MITTAL SCHOOL OF BUSINESS

Course Code:- INTM590	**Course Title:- BIG DATA ANALYTICS**
Course Instructor:- Mr. Robin Tyagi	
Academic Task No.:- 1	**Academic Task Title:** Project
Date of Allotment: 24-04-2026	**Date of submission**: 30-04-2026
Student's Roll no:- RQ5E06A15	**Student's Reg. no:-** 12510676
Evaluation Parameters:	

Learning Outcomes:-

1. Learned to create and manage internal and external tables in Hive.
2. Gained practical experience in loading local data into Hive tables.
3. Understood static and dynamic partitioning for better data organization.
4. Improved big data analytics and Hive command implementation skills.

Declaration:

I declare that this Assignment is my individual work. I have not copied it from any other student's work or from any other source except where due acknowledgement is made explicitly in the text, nor has any part been written for me by any other person.

Student's Signature: Lovish

Evaluator's comments (For Instructor's use only)

Evaluator's Signature and Date:

General Observations	Suggestions for Improvement	Best part of assignment

SUBMITTED TO: Mr. Robin Tyagi **SUBMITTED BY: Lovish**

ACKNOWLEDGEMENT

I would like to sincerely express my gratitude to my faculty member and Lovely Professional University for providing me with the opportunity to complete this project for the course Big Data Analytics. This assignment has helped me gain valuable practical knowledge and a deeper understanding of Hive, Hadoop, and data analytics tools.

I am thankful to my instructor for their continuous guidance, support, and encouragement throughout this project. Their insights and teaching played an important role in successfully completing this assignment.

I would also like to appreciate the resources and academic environment provided by the Mittal School of Business, which supported my learning process and enhanced my technical skills.

Lastly, I am grateful for this learning experience, which has strengthened my confidence in applying big data concepts practically.

CERTIFICATE

This is to certify that the project titled **"Implementation of Hive Analytics Commands Using Big Data Tools"** submitted by the student of Lovely Professional University, under the course **INTM590 – Big Data Analytics**, is an original work carried out as part of the project requirements.

The work has been completed under the guidance of the faculty member of Mittal School of Business, Lovely Faculty of Business and Arts, during the academic session as per university guidelines.

This project demonstrates the student's understanding of Hive operations, data loading techniques, and partitioning methods in big data environments.

INTRODUCTION

Big Data Analysis with Apache Hive

In today's data-driven world, organizations generate massive volumes of structured and unstructured data every day. Managing, storing, and analyzing this large-scale data efficiently requires advanced big data technologies. Apache Hive is one of the most widely used data warehouse tools in the Hadoop ecosystem that enables users to process and analyze large datasets using SQL-like queries, known as Hive Query Language (HQL).

As businesses continue to expand digitally, the need for scalable and cost-effective data processing systems has become increasingly important. Traditional database systems often struggle to manage huge datasets, whereas Hive provides a reliable framework for querying and analyzing distributed data stored in Hadoop Distributed File System (HDFS). Hive simplifies complex big data operations by offering a user-friendly interface similar to SQL, making it accessible for analysts, developers, and data engineers.

This project focuses on implementing essential Hive analytics commands to understand how data is managed and processed within a distributed environment. Through this practical assignment, important Hive operations such as creating internal tables, creating external tables, and implementing static as well as dynamic partitioning are performed. These operations are fundamental for organizing data effectively, improving storage flexibility, and optimizing query performance.

Internal tables in Hive store both table metadata and actual data within the Hive warehouse, making Hive responsible for complete data management. This means that when an internal table is deleted, both the table structure and stored data are removed. External tables, on the other hand, allow Hive to access data stored outside the warehouse location, providing better flexibility for managing large datasets without affecting the original source files. This distinction is crucial in enterprise systems where data security, backup, and shared access are important.

Partitioning techniques further enhance Hive's performance by dividing large tables into smaller, manageable segments based on specific column values. Static partitioning requires manually specifying partition values, while dynamic partitioning automatically creates partitions during data insertion. These methods improve data retrieval speed, reduce query processing time, and optimize system resources, making Hive more efficient for large-scale analytical tasks.

Additionally, learning Hive operations provides valuable insight into how modern organizations use data warehousing tools for reporting, forecasting, customer analysis, and strategic decision-making. Hive is widely used in industries such as finance, healthcare, retail, and e-commerce, where large volumes of data must be processed regularly.

The purpose of this project is to gain practical exposure to Hive's data handling capabilities while developing technical expertise in big data analytics. By performing these operations, learners can better understand real-world applications of Hive in enterprise-level data analysis, business intelligence, and large-scale data warehousing systems.

This assignment not only strengthens conceptual understanding but also builds practical skills required in modern data analytics environments. It enhances familiarity with big data frameworks, improves command-line proficiency, and prepares students for professional roles involving Hadoop, data engineering, and business analytics. Overall, this project serves as an important step toward mastering industry-relevant big data technologies and analytical tools.

Objectives of the Project

1. **To understand the practical implementation of Hive analytics commands** by performing essential database operations such as creating internal and external tables within the Hadoop ecosystem.
2. **To develop hands-on experience in loading and managing data from local systems into Hive tables**, enabling efficient data storage and retrieval for analytical purposes.
3. **To learn the differences between internal and external tables in Hive**, including their data management structure, storage behavior, and use cases in real-world big data environments.
4. **To implement static and dynamic partitioning techniques** for better organization of large datasets, improving query optimization and performance in Hive.
5. **To strengthen technical knowledge of big data tools and data warehousing concepts** by applying Hive functionalities in practical scenarios.
6. **To enhance problem-solving and analytical skills** by working with distributed data systems and understanding how Hive simplifies large-scale data analysis.

Significance of the Project

This project holds significant academic and practical value as it provides a strong foundation in big data analytics using Apache Hive. In modern organizations, managing vast amounts of data efficiently is critical for informed decision-making, and Hive plays a major role in simplifying this process.

By understanding internal and external table creation, students gain insight into how different storage methods are used based on business requirements. Internal tables are useful for complete data management, while external tables are crucial when handling shared or externally stored datasets.

The implementation of static and dynamic partitioning is particularly important because partitioning improves query execution speed, minimizes resource usage, and supports scalable

data analysis. These concepts are widely applied in industries dealing with large transactional, operational, or customer datasets.

Furthermore, this project bridges theoretical learning with practical application, preparing students for real-world roles in data analytics, business intelligence, cloud computing, and Hadoop administration. It also enhances familiarity with enterprise-level tools that are commonly used in modern big data infrastructures.

Overall, this assignment contributes to developing industry-relevant technical expertise, making it an essential learning experience for students pursuing careers in data science, analytics, and big data management.

1. Create Internal Table and Load Data

An **Internal (Managed) Table** is one where Hive manages the lifecycle of the data. If you drop the table, the data is also deleted from HDFS.

Steps:

1. **Prepare the Data:** Create a local file named sale.txt on the Desktop.
2. **Create Table:**

SQL

```
CREATE TABLE internal_sales (id INT, category STRING, amount DOUBLE, date STRING)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ',';
```

3. **Load Data:** Use the LOCAL keyword to pull data from the Linux file system into the Hive warehouse.

SQL

```
LOAD DATA LOCAL INPATH '/home/cloudera/Desktop/sale.txt' INTO TABLE internal_sales;
```

4. **Verification:** Run SELECT * FROM internal_sales; to confirm the 12 rows are loaded.

2. Create External Table and Load Data

An **External Table** points to a specific location in HDFS. If you drop the table, the metadata is removed, but the data remains in HDFS.

Steps:

1. **Create HDFS Directory:** Prepare a landing spot for the data. hdfs dfs -mkdir /user/hive/external_retail_data
2. **Create Table:** Specify the EXTERNAL keyword and the HDFS LOCATION.

SQL

CREATE EXTERNAL TABLE external_sales (id INT, category STRING, amount DOUBLE, date STRING)

ROW FORMAT DELIMITED

FIELDS TERMINATED BY ','

LOCATION '/user/hive/external_retail_data/';

3. **Load Data:**

SQL

LOAD DATA LOCAL INPATH '/home/cloudera/Desktop/sale.txt' INTO TABLE external_sales;

Note: Loading into an external table moves the file into the specified HDFS location (as seen in your image 6.6).

3. Static and Dynamic Partitioning

Partitioning divides a table into parts based on column values (like category or month), which significantly improves query performance.

Static Partitioning

In Static partitioning, you manually specify the partition value during the load process. This is ideal when you are loading a file that contains data for only one specific category or date.

- **Create Table:**

SQL

CREATE TABLE static_partition_sales (id INT, category STRING, amount DOUBLE)

PARTITIONED BY (sales_month STRING);

- **Load Command:** You must explicitly define the partition value.

SQL

LOAD DATA LOCAL INPATH '/path/to/jan_sale.txt'

INTO TABLE static_partition_sales

PARTITION (sales_month='Jan');

Dynamic Partitioning

In Dynamic partitioning, Hive automatically creates partitions based on the values in the data being inserted. This is used when moving data from one table to another.

- **Step 1: Enable Dynamic Partitioning:**

SQL

```
SET hive.exec.dynamic.partition=true;

SET hive.exec.dynamic.partition.mode=nonstrict;
```

- **Step 2: Create Partitioned Table:**

SQL

```
CREATE TABLE dynamic_partition_sales (id INT, amount DOUBLE)

PARTITIONED BY (category STRING);
```

- **Step 3: Insert Data:**

SQL

```
INSERT OVERWRITE TABLE dynamic_partition_sales PARTITION (category)

SELECT id, amount, category FROM internal_sales;
```

CONCLUSION

This project provided valuable practical exposure to the implementation of essential Hive analytics commands within the Hadoop ecosystem. By performing operations such as creating internal tables, external tables, and implementing both static and dynamic partitioning, a deeper understanding of Hive's role in big data storage, management, and analysis was developed.

Through the creation of internal tables, the project demonstrated how Hive manages both metadata and actual data within its warehouse, offering complete control over stored datasets. The implementation of external tables highlighted the flexibility of accessing data stored outside Hive's warehouse while preserving original source files, which is highly beneficial in large-scale enterprise environments. Additionally, learning static and dynamic partitioning techniques showed how large datasets can be organized more efficiently, significantly improving query performance and reducing processing time.

This assignment also strengthened technical proficiency in using Hive Query Language (HQL) for real-world data management tasks. It enhanced understanding of data warehousing concepts, distributed data processing, and optimization strategies that are widely used in business intelligence and analytics industries. Working on these practical implementations bridged the gap between theoretical knowledge and hands-on application, making big data concepts easier to understand and apply.

Overall, this project successfully improved analytical thinking, problem-solving capabilities, and technical expertise in Hive and Hadoop tools. It serves as an important foundation for future learning in data analytics, cloud computing, and large-scale data engineering. The knowledge and skills gained through this assignment are highly relevant for modern data-driven organizations and will be beneficial for academic growth as well as professional career development in the field of big data analytics.